

Slot Tagging
NLP 243
Homework 3

Task

In this homework I worked on the problem of slot tagging in the context of movie user queries. The goal of this assignment is to classify parts of speech in natural language utterances. Each word in an input sentence is assigned an IOB tag where B means the beginning of an entity, I meaning words inside an entity and O meaning outside an entity.

The dataset given consists of movie related user queries with corresponding IOB slot tags. Thus using this I decided to create a bidirectional LSTM model trained on the utterance, IOB dataset to be able to perform slot tagging on any given utterance in the context of movie queries.

Data Preparation

In order to process the data for training I made all text lowercase for more consistency. Then I used padding tokens to make all utterances the same length and UNK tokens for unknown. Using this the utterances are batched and sorted for efficient sequence processing

Model

My model is a Bidirectional Long Short Term Memory network. I chose this model because I know that LSTMs would be suitable for the task since it would be able to look at the context before any word with higher priority towards more recent words, and making it bidirectional gives more accuracy as it's able to look both forward and backward within an utterance.

The model first enters an embedding layer where it maps to a 128-dimensional vector where I added padding for variable length sequences. Then the LSTM implementation is a 2 layer BiLSTM with 256 hidden units per direction. After this I apply a dropout with 0.5. And finally goes through the output linear layer for tagging the utterance.

Hyperparameters and Experiments

The following describes the final hyperparameters that led to the best performance of the model. The model tends to converge at around 20-25 epochs and using a learning rate scheduler helped to overcome loss plateaus.

```
EMBEDDING_DIM = 128
HIDDEN_DIM = 256
NUM_LAYERS = 2
DROPOUT = 0.5
LEARNING_RATE = 0.001
BATCH_SIZE = 32
NUM_EPOCHS = 25
```

Hidden Dim	Train F1
64	~0.82
128	~0.87
256	~0.90
512	~0.89

In my experimentation first I looked at modifications in the hidden dimensionality. Values below 128 seemed to show underfitting with best results at 256 hidden dimensions. And after 256 overfitting was beginning to be noticed.

Layers	Train F1
1	~0.85
2	~0.90
3	~0.89

After this I tried to modify the number of layers in the LSTM and found that 1 layer did not allow enough capacity to learn the complex patterns and 3+ layers were showing diminishing returns with an overly complex model.

Dropout	Train F1
0.3	~0.91
0.5	~0.90
0.7	~0.86

Conclusion

Using a BiLSTM model proves to be very effective in slot tagging tasks. Allowing the model to have access to context both forward and backward in text allows it to gain a lot more context in entity boundaries. I also was able to see the causes of overfitting and diminishing returns through model complexity of adding too many layers and changing hidden dimensionality.